Mode REST standalone — Architecture et configuration

Mode REST standalone — Architecture et configuration

Les widgets EMILE fonctionnent normalement dans un **iframe Grist** (mode plugin). Le mode REST permet d'y accéder depuis une **URL publique**, sans iframe — pour accès orienteur standalone ou formulaires autonomes.

Widgets accessibles en mode REST

Widget	URL directe	Usage
`fiche-candidat`	`/widgets/emile/fiche-candidat/`	Auto-sélection du candidat le plus récent (session localStorage)
`fiche-candidat`	`/widgets/emile/fiche-candidat/?token=OCC&id=ROW_ID`	Accès orienteur à un candidat précis via token OCC
`liste-candidats`	`/widgets/emile/liste-candidats/?token=OCC`	Liste des candidats de l'orienteur via token OCC
`ajout-etablissement`	`/widgets/emile/ajout-etablissement/`	Formulaire d'ajout d'un établissement
`creation-compte-orienteur`	`/widgets/emile/creation-compte-orienteur/`	Formulaire de création d'un compte orienteur — AddRecord + génération automatique du token OCC
`inscription-candidat`	`/widgets/emile/inscription-candidat/`	Formulaire d'inscription candidat — AddRecord + génération token OCC pour l'orienteur
`validation-compte`	`/widgets/emile/validation-compte/?token=X.HMAC`	Activation d'un compte orienteur via token signé
`recuperer-lien-connexion`	`/widgets/emile/recuperer-lien-connexion/`	Envoi du lien de connexion orienteur par email
`recuperer-lien-validation`	`/widgets/emile/recuperer-lien-validation/`	Envoi du lien de validation de compte par email

Architecture

```
Navigateur (GitHub Pages, CORS bloqué vers Grist)
    │
    │  GET ?table=CANDIDATS  (ou POST text/plain pour écriture)
    ▼
https://n8n.incubateur.../webhook/grist-proxy   ← proxy
    │  vérifie HMAC-SHA256 du token
```

```
    │  Bearer <clé API>  (jamais exposée au navigateur)
    │  GET https://grist.incubateur.dnum.din.developpement-durable.gouv.fr
    │       /api/docs/75GHATRaKvHSmx3FRqCi4f/tables/CANDIDATS/records?filter={"id":[rowId]}
    ▼
Grist (instance interne)
    │  {"records": [{id: 42, fields: {...}}]}
    ▼
n8n → réponse avec Access-Control-Allow-Origin: *
    ▼
Widget → affiche / enregistre les données
```

Pourquoi un proxy ?

CORS — Les widgets tournent sur `stiiig.github.io`. Le navigateur bloque par sécurité tout appel JS vers un autre domaine (`grist.incubateur.dnum...`) sauf si ce domaine répond avec `Access-Control-Allow-Origin: *`. Grist ne le fait pas. n8n appelle Grist côté serveur (pas de CORS), et renvoie la réponse avec le bon header.

Clé API — Grist demande un Bearer token pour toute requête. Si on appelait Grist directement depuis le browser, la clé serait visible dans le JS. Avec n8n, elle reste côté serveur, jamais exposée.

Vérification du magic link — Le secret HMAC ne doit jamais apparaître dans le bundle JS. La vérification se fait entièrement dans n8n.

Activation du mode REST

Contrôlé par la variable d'environnement `NEXT_PUBLIC_GRIST_PROXY_URL` (baked dans le bundle au build Next.js).

Variable	Valeur	Obligatoire
`NEXT_PUBLIC_GRIST_PROXY_URL`	`https://n8n.incubateur.dnum.din.developpement-durable.gouv.fr/webhook/grist-proxy`	✅ — active le mode REST (proxy principal Grist)
`NEXT_PUBLIC_OCC_GENERATE_URL`	`https://n8n.incubateur.dnum.din.developpement-durable.gouv.fr/webhook/occ-generate`	✅ — génération du token OCC orienteur
`NEXT_PUBLIC_OCC_VALIDATE_URL`	`https://n8n.incubateur.dnum.din.developpement-durable.gouv.fr/webhook/occ-validate`	✅ — vérification et activation du compte orienteur
`NEXT_PUBLIC_OCC_LIST_URL`	`https://n8n.incubateur.dnum.din.developpement-durable.gouv.fr/webhook/occ-list`	✅ — liste des candidats d'un orienteur

Variable	Valeur	Obligatoire
`NEXT_PUBLIC_OCC_GET_CANDIDAT_URL`	`https://n8n.incubateur.dnum.din.developpement-durable.gouv.fr/webhook/occ-get-candidat`	✅ — fiche d'un candidat pour un orienteur
`NEXT_PUBLIC_OCC_REQUEST_LINK_URL`	`https://n8n.incubateur.dnum.din.developpement-durable.gouv.fr/webhook/occ-request-link`	✅ — renvoi lien de connexion orienteur
`NEXT_PUBLIC_OCC_REQUEST_VALIDATION_URL`	`https://n8n.incubateur.dnum.din.developpement-durable.gouv.fr/webhook/occ-request-validation-link`	✅ — renvoi lien de validation de compte

Déclarées dans `.github/workflows/deploy.yml` :

```
env:
  NEXT_PUBLIC_GRIST_PROXY URL:              ${{ secrets.NEXT_PUBLIC_GRIST_PROXY_URL }}
  NEXT_PUBLIC_OCC_GENERATE_URL:             ${{ secrets.NEXT_PUBLIC_OCC_GENERATE_URL }}
  NEXT_PUBLIC_OCC_VALIDATE_URL:             ${{ secrets.NEXT_PUBLIC_OCC_VALIDATE_URL }}
  NEXT_PUBLIC_OCC_LIST_URL:                 ${{ secrets.NEXT_PUBLIC_OCC_LIST_URL }}
  NEXT_PUBLIC_OCC_GET_CANDIDAT_URL:         ${{ secrets.NEXT_PUBLIC_OCC_GET_CANDIDAT_URL }}
  NEXT_PUBLIC_OCC_REQUEST_LINK_URL:         ${{ secrets.NEXT_PUBLIC_OCC_REQUEST_LINK_URL }}
  NEXT_PUBLIC_OCC_REQUEST_VALIDATION_URL:   ${{ secrets.NEXT_PUBLIC_OCC_REQUEST_VALIDATION_URL }}
```

Quand cette variable est définie, `src/lib/grist/init.ts` bascule automatiquement en mode `rest` et utilise `createRestDocApi()` au lieu du plugin Grist.

Fichiers clés

`src/lib/grist/rest.ts`

Implémente `GristDocAPI` via le proxy n8n :

Méthode	Requête vers n8n	Traduite par n8n en	Workflow
`fetchTable(tableId)`	`GET ?table=TABLE`	`GET /tables/TABLE/records`	`grist-proxy-get`
`getAttachmentDownloadUrl(id)`	`GET ?attachId=ID`	`GET /attachments/ID/download`	`grist-proxy-get`
`uploadAttachments(files)`	`POST` `multipart/form-data` (champ `upload`)	`POST /attachments`	`grist-proxy-post`

Méthode	Requête vers n8n	Traduite par n8n en	Workflow
`applyUserActions([["AddRecord", ...]])`	`POST text/plain { _action:"add", fields }`	`POST /tables/TABLE/records`	`grist-proxy-post`
`applyUserActions([["UpdateRecord", ...]])`	`POST text/plain { _action:"update", id, fields }`	`PATCH /tables/TABLE/records/{id}`	`grist-proxy-post`

> `text/plain;charset=UTF-8` est une *simple CORS request* (spec WHATWG Fetch) — pas de preflight OPTIONS. Le champ `_action` permet à n8n de distinguer AddRecord d'UpdateRecord dans le même workflow POST.

`src/lib/grist/meta.ts` — `loadColumnsMetaFor`

Charge les métadonnées des colonnes (types, options, choices) via les tables internes Grist : 1. `fetchTable("_grist_Tables")` → trouve le rowId de la table cible 2. `fetchTable("_grist_Tables_column")` → récupère les colonnes de cette table

Ces deux tables sont accessibles via le proxy n8n comme n'importe quelle table normale (pas de token nécessaire).

`src/lib/grist/hooks.ts`

- `useGristInit` : charge `grist-plugin-api.js` uniquement si on est dans un iframe. En mode standalone, ce script n'est jamais chargé.
- `useDepartementOptions` : charge `DPTS_REGIONS` via `fetchTable`, retourne `{deptOptions, dptsLoading, dptsError}`.

Configuration n8n

Workflows actifs

Workflow n8n	Méthode	Path	Usage
`grist-proxy-get`	GET	`/webhook/grist-proxy`	Lecture records, métadonnées, téléchargement PJ
`grist-proxy-post`	POST	`/webhook/grist-proxy`	Écritures Grist (AddRecord/UpdateRecord), upload pièces jointes
`occ-generate`	GET	`/webhook/occ-generate`	Génération du token OCC orienteur
`occ-validate`	GET	`/webhook/occ-validate`	Vérification token OCC + activation compte orienteur
`occ-list`	GET	`/webhook/occ-list`	Liste des candidats d'un orienteur (token OCC)
`occ-get-candidat`	GET	`/webhook/occ-get-candidat`	Fiche d'un candidat pour un orienteur (token OCC)
`occ-request-link`	POST	`/webhook/occ-request-link`	Renvoi du lien de connexion orienteur par email

Workflow n8n	Méthode	Path	Usage
`occ-request-validation-link`	POST	`/webhook/occ-request-validation-link`	Renvoi du lien de validation de compte par email

Archivé : `grist-generate` (magic link candidat — remplacé par token OCC orienteur).

> ℹ️ `grist-proxy-get` et `grist-proxy-post` partagent le même path webhook `/webhook/grist-proxy`. n8n route automatiquement selon la méthode HTTP. Le code n'utilise qu'une seule variable d'environnement `NEXT_PUBLIC_GRIST_PROXY_URL` pour les deux.

Workflow `grist-proxy-get` — records + métadonnées + téléchargement (GET)

URL (production) :

```
https://n8n.incubateur.dnum.din.developpement-durable.gouv.fr/webhook/grist-proxy
```

> ℹ️ Ancien nom : `grist - GET fiche-candidat`, ancien path : `/webhook/grist`. Renommé et déplacé vers `/webhook/grist-proxy` lors de la refonte du proxy.

Nœud 1 — Webhook (GET)

Paramètre	Valeur
HTTP Method	**GET**
Path	`grist-proxy`
Response Mode	Using Respond to Webhook Node

Nœud 2 — IF token présent (usage interne legacy)

Paramètre	Valeur
Condition	`{{ $json.query.token }}` **is not empty**
True →	branche vérification HMAC
False →	**IF attachId** (chemin classique, pas de 403 !)

> ⚠️ **Piège critique** : la branche **faux** doit aller vers **IF attachId**, jamais vers un Respond 403. Les requêtes de métadonnées (`_grist_Tables`, `_grist_Tables_column`, etc.) n'ont pas de token — les rejeter avec 403 empêche le widget de charger les colonnes et les dropdowns.

Branche token (True)

Nœud 3a — Code (extraction rowId + sig)

```
const token = $json.query.token;        // ex: "42.a3f9b2..."
const parts = token.split(".");
```

```
const rowId = parts[0];
const sig   = parts[1];
if (!rowId || !sig || isNaN(Number(rowId))) throw new Error("Token malformé");
return [{ json: { rowId: Number(rowId), sig, token } }];
```

Nœud 4a — Crypto (HMAC-SHA256)

Paramètre	Valeur
Action	**HMAC**
Type	**SHA256**
Value	`{{ $json.rowId.toString() }}`
Credential	**EMILE HMAC Secret** (credential Crypto n8n)
Encoding	**HEX**
Property Name	`data`

Nœud 5a — IF signature valide

Paramètre	Valeur
Value 1	`{{ $json.data }}` (HMAC calculé)
Operation	**equals**
Value 2	`{{ $json.sig }}` (sig extrait du token)
True →	IF attachId (lecture normale)
False →	Respond to Webhook 403 `{ "error": "Token invalide" }`

Branche classique (False du IF token) + branche token valide

Les deux chemins convergent vers :

Nœud — IF attachId

Paramètre	Valeur
Condition	`{{ $json.query.attachId }}` **is not empty**
True →	branche téléchargement pièce jointe
False →	branche records

> ⚠️ Activer **"Convert types where required"** dans ce nœud IF.

Branche download (True)

HTTP Request (fichier binaire)

Paramètre	Valeur
Method	GET
URL	`https://grist.incubateur.dnum.din.developpement-durable.gouv.fr/api/docs/75GHATRaKvHSmx3FRqCi4f/attachments/{{ $json.query.attachId }}/download`
Authentication	Generic Credential Type → Bearer Auth → **Bearer Auth Grist**
Response Format	**File**

> ⚠️ Sans `Response Format: File`, n8n tente de parser la réponse comme JSON et échoue.

Respond to Webhook (binaire)

Paramètre	Valeur
Respond With	**Binary Data**
Response Data Property Name	`data`
Response Code	200
Header	`Access-Control-Allow-Origin: *`

Branche records (False)

HTTP Request (appel Grist — records)

Paramètre	Valeur
Method	GET
Authentication	Generic Credential Type → Bearer Auth → **Bearer Auth Grist**

URL (expression, sans activer `fx` séparément) :

```
https://grist.incubateur.dnum.din.developpement-durable.gouv.fr/api/docs/75GHATRaKvHSmx3FRqCi4f/tables/{{ $json.rowId ? 'CANDIDATS' : $json.query.table }}/records{{ $json.rowId ? '?filter=' + JSON.stringify({id: [$json.rowId]}) : ($json.query.filter ? '?filter=' + $json.query.filter : '') }}
```

Cette expression gère les deux cas : - **Token path** (`$json.rowId` défini) → table = `CANDIDATS`, filter = `{"id": [rowId]}` - **Chemin classique** (`$json.query` intact) → table et filter viennent de la query

> ⚠️ Ne jamais ajouter de param `filter` dans “Using Fields Below” — utiliser uniquement l’expression dans l’URL.

Cas couverts :

Requête entrante	URL envoyée à Grist
`?table=ETABLISSEMENTS`	`.../tables/ETABLISSEMENTS/records`
`?table=CANDIDATS&filter={"id":[42]}`	`.../tables/CANDIDATS/records?filter=...`

Requête entrante	URL envoyée à Grist
`?table=_grist_Tables`	`.../tables/_grist_Tables/records`
`?table=CANDIDATS&token=42.abc...`	`.../tables/CANDIDATS/records?filter={"id":[42]}`

Respond to Webhook (JSON)

Paramètre	Valeur
Respond With	JSON
Response Code	200
Response Body	`{{ $json }}`
Header	`Access-Control-Allow-Origin: *`

> ⚠️ Écrire `{{ $json }}` et **non** `={{ $json }}`.

Schéma — Workflow `grist-proxy-get`

```
Webhook GET (path: grist-proxy)
    │
    ▼
IF query.token is not empty
    │
    ├─ True ──► Code (extrait rowId + sig)
    │           └──► Crypto HMAC-SHA256 (credential EMILE HMAC Secret)
    │                └──► IF data === sig
    │                     ├─ True  ──► IF attachId (voir ci-dessous)
    │                     └─ False ──► Respond 403 { "error": "Token invalide" }
    │
    └─ False ──► IF query.attachId is not empty  [Convert types: ON]
                  ├─ True  ──► HTTP GET /attachments/{id}/download (Format: File)
                  │             └──► Respond Binary Data + CORS header
                  └─ False ──► HTTP GET /tables/{table}/records (URL expression)
                                └──► Respond JSON (body: {{ $json }}) + CORS header
```

Workflow `grist-proxy-post` — écritures Grist (AddRecord/UpdateRecord) + upload de pièces jointes (POST)

URL (production) :

```
https://n8n.incubateur.dnum.din.developpement-durable.gouv.fr/webhook/grist-proxy
```

(même path que `grist-proxy-get` — n8n route par méthode HTTP)

> ℹ️ Ancien nom : `grist - POST fiche-candidat`, ancien path : `/webhook/grist`. Renommé et déplacé vers

> `/webhook/grist-proxy` lors de la refonte du proxy.

> ℹ️ Le workflow POST reçoit deux types de requêtes, toutes deux sans preflight CORS OPTIONS : - `text/plain;charset=UTF-8` (écritures Grist — AddRecord/UpdateRecord) - `multipart/form-data` (upload de pièces jointes)
> Un nœud IF au début du workflow route vers la bonne branche selon le Content-Type.

Nœud 1 — Webhook (POST)

Paramètre	Valeur
HTTP Method	**POST**
Path	`grist-proxy`
Response Mode	Using Respond to Webhook Node

Nœud 2 — IF Content-Type text/plain (écritures)

Paramètre	Valeur
Condition	`{{ $json.headers["content-type"] }}` **contains** `text/plain`
True →	branche AddRecord/UpdateRecord
False →	branche upload (multipart)

Branche True — écritures (text/plain)

Nœud 3a — Code (parser le body text/plain)

```
// rest.ts envoie du JSON sérialisé en texte brut (Content-Type: text/plain)
// n8n ne parse pas automatiquement text/plain — on le fait manuellement ici
const raw = $json.body;
const parsed = JSON.parse(raw);             // { _action, id?, fields }

// La table cible est dans le query param (ex: ?table=CANDIDATS)
const table  = $json.query.table;
const action = parsed._action;             // "add" ou "update"
const id     = parsed.id ?? null;
const fields = parsed.fields ?? {};

return [{ json: { table, action, id, fields } }];
```

> ⚠️ **Piège** : ne pas essayer d'interpoler `{{ $json.fields }}` directement dans un body JSON n8n — un objet JavaScript ne peut pas être injecté comme valeur d'un champ de template JSON. Il faut passer par un nœud Code qui fait `JSON.stringify`, puis utiliser un body **Raw** (voir nœud suivant).

Nœud 4a — IF action (add vs update)

Paramètre	Valeur
Condition	`{{ $json.action }}` **equals** `update`
True →	branche UpdateRecord (PATCH)
False →	branche AddRecord (POST)

Sous-branche True (Nœud 4a) — UpdateRecord

Nœud 5a-update — Code (construire le body PATCH)

```
const id     = $json.id;
const fields = $json.fields;
const body   = JSON.stringify({ records: [{ id, fields }] });
return [{ json: { ...$json, gristBody: body } }];
```

Nœud 6a-update — HTTP Request (UpdateRecord → PATCH Grist)

Paramètre	Valeur
Method	**PATCH**
URL	`https://grist.incubateur.dnum.din.developpement-durable.gouv.fr/api/docs/75GHATRaKvHSmx3FRqCi4f/tables/{{ $json.table }}/records`
Authentication	Generic Credential Type → Bearer Auth → **Bearer Auth Grist**
Body Content Type	**Raw**
Content Type	`application/json`
Body	`{{ $json.gristBody }}`

> ℹ️ n8n appelle Grist en PATCH côté serveur — il n'y a pas de contrainte CORS ici. La requête browser → n8n est un `POST text/plain` (simple CORS), c'est n8n qui fait le PATCH vers Grist.

Nœud 7a-update — Respond to Webhook (UpdateRecord)

Paramètre	Valeur
Respond With	**JSON**
Response Body	`{}`
Response Code	200
Header	`Access-Control-Allow-Origin: *`

Sous-branche False (Nœud 4a) — AddRecord

Nœud 5a-add — Code (construire le body POST)

```
const fields = $json.fields;
const body   = JSON.stringify({ records: [{ fields }] });
return [{ json: { ...$json, gristBody: body } }];
```

Nœud 6a-add — HTTP Request (AddRecord → POST Grist)

Paramètre	Valeur
Method	**POST**
URL	`https://grist.incubateur.dnum.din.developpement-durable.gouv.fr/api/docs/75GHATRaKvHSmx3FRqCi4f/tables/{{ $json.table }}/records`
Authentication	Generic Credential Type → Bearer Auth → **Bearer Auth Grist**
Body Content Type	**Raw**
Content Type	`application/json`
Body	`{{ $json.gristBody }}`

Nœud 7a-add — Code (formater la réponse)

```
// Grist renvoie { records: [{ id: 42, fields: {} }] }
// rest.ts attend { retValues: [42] }
const records = $json.records ?? [];
const newId = records[0]?.id ?? null;
return [{ json: { retValues: [newId] } }];
```

Nœud 8a-add — Respond to Webhook (AddRecord)

Paramètre	Valeur
Respond With	**JSON**
Response Body	`{{ $json }}`
Response Code	200
Header	`Access-Control-Allow-Origin: *`

Branche False (Nœud 2) — Upload de pièces jointes (multipart/form-data)

Nœud 3b — HTTP Request (upload vers Grist)

Paramètre	Valeur
Method	**POST**
URL	`https://grist.incubateur.dnum.din.developpement-durable.gouv.fr/api/docs/75GHATRaKvHSmx3FRqCi4f/attachments`
Authentication	Generic Credential Type → Bearer Auth → **Bearer Auth Grist**
Body Content Type	**Form-Data**

Paramètre	Valeur
Body / Name	`upload`
Body / Type	**n8n Binary File**
Body / Input Data Field Name	`upload`

> ⚠️ Le champ **Name** doit être `upload` (pas `data`). C'est le nom du champ multipart envoyé à Grist.

> ⚠️ Sans l'**Authentication** configurée sur ce nœud, Grist renvoie une page HTML 403 — n8n lève "Invalid JSON in response body".

Nœud 4b — Respond to Webhook (upload)

Paramètre	Valeur
Respond With	JSON
Response Body	`{{ $json }}`
Response Code	200
Header	`Access-Control-Allow-Origin: *`

Schéma complet — Workflow `grist-proxy-post`

```
Webhook POST (path: grist-proxy)
    │
    ▼
IF content-type contains "text/plain"         [Noeud 2]
    │
    ├─ True ──► Code (parse body → table, action, id, fields)   [Noeud 3a]
    │           └──► IF action equals "update"                   [Noeud 4a]
    │                 │
    │                 ├─ True  ──► Code (build PATCH body)       [Noeud 5a-update]
    │                 │            └──► HTTP PATCH /tables/{table}/records  [Noeud 6a-update]
    │                 │                 └──► Respond JSON {} + CORS         [Noeud 7a-update]
    │                 │
    │                 └─ False ──► Code (build POST body)        [Noeud 5a-add]
    │                              └──► HTTP POST /tables/{table}/records   [Noeud 6a-add]
    │                                   └──► Code { retValues: [newId] }    [Noeud 7a-add]
    │                                        └──► Respond JSON { retValues } + CORS  [Noeud 8a-add]
    │
    └─ False ──► HTTP POST /attachments (Form-Data)              [Noeud 3b]
                 └──► Respond JSON {{ $json }} + CORS            [Noeud 4b]
```

~~Workflow GENERATE — génération de magic links~~ (ARCHIVÉ)

> ⚠️ **Ce workflow est archivé.** Le mode “magic link candidat” a été remplacé par le token OCC orienteur. Le workflow `grist-generate` peut être désactivé dans n8n — il n’est plus appelé par aucun widget.
> La documentation ci-dessous est conservée à titre de référence historique.

URL (ancienne) :

```
https://n8n.incubateur.dnum.din.developpement-durable.gouv.fr/webhook/grist-generate
```

Appelé automatiquement par le formulaire `inscription-candidat` après la création d’un candidat (`GET ?rowId=X`). Peut aussi être appelé manuellement depuis un script ou une automation Grist.

Pas d’authentification — le formulaire appelle ce GET sans header custom (pas de preflight CORS). La sécurité repose sur l’obscurité de l’URL et l’impossibilité de forger un token HMAC sans le secret.

Nœud 1 — Webhook (GET)

Paramètre	Valeur
HTTP Method	**GET**
Path	`grist-generate`
Authentication	**None**
Respond	**Using Respond to Webhook Node**

> ℹ️ Anciennement POST + Basic Auth. Changé en GET pour permettre l’appel depuis le navigateur sans preflight CORS (un GET sans header custom = *simple CORS request*).

Nœud 2 — Code (extraire rowId)

```
// rowId arrive en query param : GET /webhook/grist-generate?rowId=42
const rowId = parseInt($json.query.rowId);
if (!rowId || isNaN(rowId)) throw new Error("rowId requis");
return [{ json: { rowId, rowIdStr: rowId.toString() } }];
```

Nœud 3 — Crypto (HMAC-SHA256)

Paramètre	Valeur
Action	**HMAC**
Type	**SHA256**
Value	`{{ $json.rowIdStr }}`
Credential	**EMILE HMAC Secret** (même credential que le workflow GET !)
Encoding	**HEX**
Property Name	`data`

Nœud 4 — Code (construire token + URL)

```
const rowId = $json.rowId;
const sig   = $json.data;
const token = `${rowId}.${sig}`;
const url   = `https://stiiig.github.io/grist-widgets/widgets/emile/fiche-candidat?token=${token}`;
return [{ json: { rowId, token, url } }];
```

Nœud 5 — Respond to Webhook

Paramètre	Valeur
Respond With	**First Incoming Item**

Retourne `{ rowId, token, url }`. Le header `Access-Control-Allow-Origin: *` est ajouté automatiquement par n8n sur les webhooks GET.

Nœud 6 — Code (construire body PATCH pour Lien_acces)

```
// Sauvegarde du lien dans la table CANDIDATS — exécuté APRÈS Respond to Webhook
// n8n continue son exécution après avoir répondu au browser
const rowId = $json.rowId;
const url   = $json.url;
const body  = JSON.stringify({ records: [{ id: rowId, fields: { Lien_acces: url } }] });
return [{ json: { ...$json, patchBody: body } }];
```

Nœud 7 — HTTP Request (PATCH Grist — sauvegarde Lien_acces)

Paramètre	Valeur
Method	**PATCH**
URL	`https://grist.incubateur.dnum.din.developpement-durable.gouv.fr/api/docs/75GHATRaKvHSmx3FRqCi4f/tables/CANDIDATS/records`
Authentication	Generic Credential Type → Bearer Auth → **Bearer Auth Grist**
Body Content Type	**Raw**
Content Type	`application/json`
Body	`{{ $json.patchBody }}`

> ℹ️ Ce nœud s'exécute **après** le Respond to Webhook (Nœud 5). n8n continue l'exécution du workflow après avoir répondu au browser — le frontend reçoit immédiatement `{rowId, token, url}` sans attendre la mise à jour Grist. La sauvegarde du lien dans Grist est donc non bloquante du point de vue du browser.
> ℹ️ Le frontend (`inscription-candidat`) sauvegarde également `Lien_acces` via `UpdateRecord` après réception de la réponse — il y a donc deux tentatives de sauvegarde (n8n en background + frontend via proxy POST). Les deux sont non bloquantes et idempotentes.

Exemple d'appel manuel

```
curl "https://n8n.incubateur.dnum.din.developpement-durable.gouv.fr/webhook/grist-generate?rowId=42"
# → { "rowId": 42, "token": "42.a3f9b2...", "url": "https://stiiig.github.io/...?token=42.a3f9b2..." }
```

Workflow OCC-GENERATE — génération de magic links orienteur

URL (production) :

```
https://n8n.incubateur.dnum.din.developpement-durable.gouv.fr/webhook/occ-generate
```

OCC = **Orienteur Compte Créer**. Ce workflow génère un lien signé envoyé par email à l'orienteur pour valider son compte. Il est appelé automatiquement par le formulaire `creation-compte-orienteur` après la création du compte (AddRecord dans `ACCOMPAGNANTS`).

> ℹ️ Ce workflow est **indépendant** du workflow GENERATE (fiche-candidat). Il utilise le même mécanisme HMAC-SHA256 et le même credential secret, mais pointe vers une URL de destination différente (`validation-compte` au lieu de `fiche-candidat`).

Pas d'authentification — GET sans header custom = simple CORS request. La sécurité repose sur l'obscurité de l'URL et l'impossibilité de forger un token HMAC sans le secret.

Nœud 1 — Webhook (GET)

Paramètre	Valeur
HTTP Method	**GET**
Path	`occ-generate`
Authentication	**None**
Respond	**Using Respond to Webhook Node**

Nœud 2 — Code (extraire rowId)

```
const rowId = parseInt($json.query.rowId);
if (!rowId || isNaN(rowId)) throw new Error("rowId requis");
return [{ json: { rowId, rowIdStr: rowId.toString() } }];
```

Nœud 3 — Crypto (HMAC-SHA256)

Paramètre	Valeur
Action	**HMAC**
Type	**SHA256**
Value	`{{ $json.rowIdStr }}`
Credential	**EMILE HMAC Secret** (même credential que GENERATE et OCC-VALIDATE !)
Encoding	**HEX**
Property Name	`data`

Nœud 4 — Code (construire token + URL)

```
const rowId = $json.rowId;
const sig   = $json.data;
const token = `${rowId}.${sig}`;
const url   = `https://stiiig.github.io/grist-widgets/widgets/emile/validation-compte?token=${token}`;
return [{ json: { rowId, token, url } }];
```

> ℹ️ La seule différence avec GENERATE est l'URL de destination : `validation-compte` au lieu de `fiche-candidat`.

Nœud 5 — Respond to Webhook

Paramètre	Valeur
Respond With	**First Incoming Item**

Retourne `{ rowId, token, url }`.

Nœud 6 — Code (construire body PATCH pour Lien_validation)

```
// Sauvegarde du lien dans la table ACCOMPAGNANTS — exécuté APRÈS Respond to Webhook
const rowId = $json.rowId;
const url   = $json.url;
const body  = JSON.stringify({ records: [{ id: rowId, fields: { Lien_validation: url } }] });
return [{ json: { ...$json, patchBody: body } }];
```

Nœud 7 — HTTP Request (PATCH Grist — sauvegarde Lien_validation)

Paramètre	Valeur
Method	**PATCH**
URL	`https://grist.incubateur.dnum.din.developpement-durable.gouv.fr/api/docs/75GHATRaKvHSmx3FRqCi4f/tables/ACCOMPAGNANTS/records`
Authentication	Generic Credential Type → Bearer Auth → **Bearer Auth Grist**
Body Content Type	**Raw**
Content Type	`application/json`
Body	`{{ $json.patchBody }}`

> ℹ️ Même pattern que dans GENERATE : Respond to Webhook d'abord, PATCH Grist ensuite en background. Le frontend reçoit immédiatement `{rowId, token, url}` sans attendre la mise à jour Grist.
> ⚠️ La colonne `Lien_validation` doit exister dans la table `ACCOMPAGNANTS` de Grist. La créer manuellement si elle n'existe pas (type Texte).

Exemple d'appel manuel

```
curl "https://n8n.incubateur.dnum.din.developpement-durable.gouv.fr/webhook/occ-generate?rowId=6"
# → { "rowId": 6, "token": "6.d4f8a2...", "url": "https://stiiig.github.io/.../validation-compte?
        token=6.d4f8a2..." }
```

Workflow OCC-VALIDATE — vérification et activation du compte orienteur

URL (production) :

```
https://n8n.incubateur.dnum.din.developpement-durable.gouv.fr/webhook/occ-validate
```

Ce workflow est appelé par la page `validation-compte` quand un orienteur clique sur son lien de validation. Il : 1. Vérifie la signature HMAC du token 2. Lit le compte dans `ACCOMPAGNANTS` (via son rowId) 3. Vérifie le champ `Compte_valide` (valeurs : `"En attente"` / `"Oui"`) 4. Si pas encore validé : met à jour `Compte_valide → "Oui"` via PATCH 5. Retourne un JSON de statut à la page frontend

Pas d'authentification — GET sans header custom = simple CORS request.

Nœud 1 — Webhook (GET)

Paramètre	Valeur
HTTP Method	**GET**
Path	`occ-validate`
Authentication	**None**
Respond	**Using Respond to Webhook Node**

Nœud 2 — Code (extraire rowId + sig du token)

```
const token = $json.query.token;
if (!token) throw new Error("Token manquant");
const parts = token.split(".");
const rowId = parseInt(parts[0]);
const sig   = parts[1];
if (!rowId || isNaN(rowId) || !sig) throw new Error("Token malformé");
return [{ json: { rowId, sig, rowIdStr: rowId.toString() } }];
```

Nœud 3 — Crypto (HMAC-SHA256)

Paramètre	Valeur
Action	**HMAC**
Type	**SHA256**
Value	`{{ $json.rowIdStr }}`
Credential	**EMILE HMAC Secret** (même credential que OCC-GENERATE !)
Encoding	**HEX**
Property Name	`data`

Nœud 4 — IF signature valide

Paramètre	Valeur
Value 1	`{{ $json.data }}` (HMAC calculé)
Operation	**equals**
Value 2	`{{ $json.sig }}` (sig extrait du token)
True →	HTTP GET Grist (lire le compte)
False →	Respond 200 `{ "status": "invalid" }`

> ℹ️ On retourne 200 même en cas de token invalide (pas de 403) — la page frontend distingue les cas via le champ `status` dans le JSON, pas via le code HTTP.

Nœud 5 (True) — HTTP Request (GET Grist — lire ACCOMPAGNANTS)

Paramètre	Valeur
Method	**GET**
URL	`https://grist.incubateur.dnum.din.developpement-durable.gouv.fr/api/docs/75GHATRaKvHSmx3FRqCi4f/tables/ACCOMPAGNANTS/records?filter={"id":[{{ $json.rowId }}]}`
Authentication	Generic Credential Type → Bearer Auth → **Bearer Auth Grist**

Nœud 6 — Code (extraire le compte + détecter déjà validé)

```
const records = $json.records ?? [];
if (!records.length) return [{ json: { status: "invalid" } }];

const record        = records[0];
const compteValide  = record.fields?.Compte_valide ?? "";
const nom           = record.fields?.Nom_de_famille ?? record.fields?.Nom ?? "";
const rowId         = record.id;

if (compteValide === "Oui") {
  return [{ json: { status: "already_validated", rowId, nom } }];
}
return [{ json: { status: "pending", rowId, nom } }];
```

Nœud 7 — IF déjà validé

Paramètre	Valeur
Condition	`{{ $json.status }}` **equals** `already_validated`
True →	Respond 200 `{ "status": "already_validated" }`
False →	Code (construire body PATCH)

Nœud 8 (False) — Code (construire body PATCH Compte_valide)

```
const rowId = $json.rowId;
const nom   = $json.nom;
const body  = JSON.stringify({ records: [{ id: rowId, fields: { Compte_valide: "Oui" } }] });
return [{ json: { rowId, nom, patchBody: body } }];
```

Nœud 9 — HTTP Request (PATCH Grist — activer le compte)

Paramètre	Valeur
Method	**PATCH**
URL	`https://grist.incubateur.dnum.din.developpement-durable.gouv.fr/api/docs/75GHATRaKvHSmx3FRqCi4f/tables/ACCOMPAGNANTS/records`
Authentication	Generic Credential Type → Bearer Auth → **Bearer Auth Grist**
Body Content Type	**Raw**
Content Type	`application/json`
Body	`{{ $json.patchBody }}`

Nœud 10 — Respond to Webhook (activation réussie)

Paramètre	Valeur
Respond With	**JSON**
Response Body	`{ "status": "ok", "nom": "{{ $json.nom }}" }`
Response Code	200
Header	`Access-Control-Allow-Origin: *`

Schéma complet — Workflow OCC-VALIDATE

```
Webhook GET (path: occ-validate)
    │
    ▼
Code (extrait rowId + sig du token)
    │
    ▼
Crypto HMAC-SHA256 (credential EMILE HMAC Secret)
    │
    ▼
IF data === sig
    │
    ├─ False ──► Respond 200 { "status": "invalid" }
    │
    └─ True  ──► HTTP GET /tables/ACCOMPAGNANTS/records?filter={"id":[rowId]}
                   │
                   ▼
                 Code (extrait enregistrement + détecte already_validated)
                   │
                   ▼
```

```
                IF status === "already_validated"
                  │
                  ├─ True  ──► Respond 200 { "status": "already_validated" }
                  │
                  └─ False ──► Code (build PATCH body)
                                │
                                ▼
                               HTTP PATCH /tables/ACCOMPAGNANTS/records
                                │
                                ▼
                               Respond 200 { "status": "ok", "nom": "..." }
```

Réponses JSON possibles

`status`	Signification	Action côté frontend
`"ok"`	Compte activé avec succès	Affiche “Compte activé !” + bouton inscription candidat
`"already_validated"`	Compte déjà activé	Affiche “Compte déjà activé” + bouton inscription candidat
`"invalid"`	Token invalide ou expiré	Affiche “Lien invalide”

Champs Grist concernés (table ACCOMPAGNANTS)

Champ	Type	Valeurs	Description
`Compte_valide`	Texte (choix)	`"En attente"` / `"Oui"`	Statut du compte. Initialisé à `"En attente"` à la création, passé à `"Oui"` par OCC-VALIDATE.
`Lien_validation`	Texte	URL complète	Magic link envoyé par email à l’orienteur. Sauvegardé par OCC-GENERATE (background PATCH).

> ⚠️ Ces deux colonnes doivent exister dans la table `ACCOMPAGNANTS` de Grist. Les créer manuellement si elles n’existent pas (type Texte).

Pièges rencontrés — référence complète

n8n — Webhook

Problème	Cause	Solution
Pas de “Any Method” dans le dropdown	Version n8n limitée	Trois workflows séparés GET, POST, GENERATE
Condition IF lève “Wrong type: ‘9’ is a string but was expecting a boolean”	Le query param est une string	Activer **“Convert types where required”** dans le nœud IF
`{"code":0,"message":"Unused Respond to Webhook node found in the workflow"}`	Le nœud Respond to Webhook n’est pas câblé	Le connecter au dernier nœud, ou passer Respond à “When Last Node Finishes”

n8n — IF et routing

Problème	Cause	Solution
Toutes les requêtes de métadonnées retournent 403	Branche faux du IF token connectée au Respond 403	La brancher sur **IF attachId** — les métadonnées n'ont jamais de token
Token invalide alors que le secret est correct	Les deux Crypto nodes utilisent des credentials différents	Vérifier que GENERATE et GET utilisent **le même credential**
`sig` tronqué dans n8n (ex: `887512` au lieu de 64 chars)	URL du magic link copiée/collée tronquée	Copier l'URL complète depuis la réponse JSON du workflow GENERATE
IF action toujours False — UpdateRecord passe en branche AddRecord	Code (parse) construisait `requestBody` (body Grist complet) sans jamais extraire `action` → `$json.action` toujours `undefined`	Code (parse) doit retourner `{ table, action, id, fields }` **sans** construire le body — déléguer la construction du body aux nœuds Code dans chaque branche (5a-update / 5a-add)

n8n — HTTP Request

Problème	Cause	Solution
"Invalid JSON in response body" sur l'upload	Authentication manquante → Grist renvoie du HTML 403	Configurer Bearer Auth Grist **sur ce nœud spécifiquement**
"Invalid JSON in response body" sur le download	Response Format = JSON alors que Grist renvoie du binaire	Passer Response Format à **File**
Champ **Name** = `data` au lieu de `upload`	Confusion entre le nom du champ Grist et la propriété n8n	Name = `upload`, Input Data Field Name = `upload`
"JSON parameter needs to be valid JSON" sur AddRecord	`{{ $json.fields }}` est un objet JS — ne peut pas être interpolé dans un template JSON n8n	Passer par un nœud Code qui fait `JSON.stringify({ records: [{ fields }] })`, puis body **Raw** dans l'HTTP Request
GENERATE retourne 404 après migration GET	Le workflow était encore configuré en POST	Changer la méthode du webhook GENERATE de **POST** à **GET**, supprimer l'Authentication
`rowId` est `NaN` dans GENERATE	Code node lit encore `$json.body.rowId` (ancien format POST)	Lire `parseInt($json.query.rowId)` — le rowId arrive en query param avec GET

n8n — Respond to Webhook

Problème	Cause	Solution
Réponse vide ou non-JSON reçue par le browser	`={{ $json }}` (syntaxe expression) au lieu de `{{ $json }}`	Écrire `{{ $json }}` **sans** le `=` préfixe

Problème	Cause	Solution
Le fichier s'ouvre dans un onglet au lieu de se télécharger	n8n ne renvoie pas `Content-Disposition: attachment` systématiquement	Le code utilise `fetch` + `blob` + `<a download>` côté browser

n8n — Format de réponse imprévisible (upload)

Grist renvoie `[42]` (tableau JSON). n8n transforme cette réponse de manière non déterministe :

Format reçu par le browser	Quand
`{"data": "[42]"}`	HTTP Request sans Code node
`{"ids": [{"json": 42, "pairedItem": {...}}]}`	Avec Code node retournant les items n8n complets
`[42]`	Certaines versions de n8n
`42`	n8n unwrappe le tableau d'un seul élément

Le code `uploadAttachmentsRest` dans `rest.ts` gère tous ces formats — **ne pas simplifier**.

n8n — OCC-VALIDATE

Problème	Cause	Solution
`rowId: null` dans le Code PATCH (token valide mais Grist renvoie des records vides)	Branches True/False du nœud IF signature inversées — le flux valide allait vers "Respond invalid" et le flux invalide vers la requête GET	Vérifier le sens des branches IF (True = valide → GET Grist, False = invalide → Respond invalid)
`"JSON parameter needs to be valid JSON"` dans le nœud HTTP PATCH	`{{ $json.url }}` interpolé dans un template JSON n8n — les URLs avec `://` et `?` brisent le parser	Passer par un nœud Code qui fait `JSON.stringify({ records: [...] })`, puis HTTP Request en mode **Raw body** avec `{{ $json.patchBody }}`
`"Invalid column Lien_validation"` dans le PATCH Grist	La colonne n'existait pas encore dans `ACCOMPAGNANTS`	Créer manuellement la colonne `Lien_validation` (type Texte) dans la table `ACCOMPAGNANTS` via l'interface Grist
La page frontend reçoit une réponse vide ou la réponse Grist au lieu de `{status, nom}`	`Respond to Webhook` placé **après** le HTTP PATCH — n8n répond avec la réponse du PATCH Grist, pas avec `{status: "ok"}`	Mettre le `Respond to Webhook` **avant** le HTTP PATCH. n8n continue l'exécution après avoir répondu.
`Content-Type:` avec deux-points dans le nom de l'en-tête du HTTP Request	Faute de frappe — le nom du header était `Content-Type:` au lieu de `Content-Type`	Corriger le nom du header (sans le `:` final)

Code — React

Problème	Cause	Solution
“Application error: a client-side exception” sur les pages avec attachments	`return null` conditionnel avant un `useCallback` → violation Rules of Hooks	Déplacer tous les `return` conditionnels **après** tous les hooks

---

~~Générer un magic link fiche-candidat~~ (ARCHIVÉ)

> ⚠️ Ce mode d’accès est archivé. L’accès à `fiche-candidat` se fait désormais exclusivement via **token OCC orienteur** (`?token=OCC&id=ROW_ID` ou session localStorage). Le workflow `grist-generate` peut être désactivé.

Générer un magic link orienteur (OCC)

Automatiquement (depuis le formulaire de création de compte)

Le formulaire `creation-compte-orienteur` génère le magic link OCC automatiquement après la création du compte (`NEXT_PUBLIC_OCC_GENERATE_URL`). Le lien de validation est visible dans l’interface de confirmation.

En parallèle, le workflow OCC-GENERATE sauvegarde le lien dans le champ `Lien_validation` de la table `ACCOMPAGNANTS` via un HTTP PATCH Grist (en background).

Le formulaire `inscription-candidat` peut aussi regénérer un lien OCC pour un orienteur existant si son compte n’est pas encore validé (`Compte_valide !== "Oui"`).

Manuellement (curl)

```
# Générer un token OCC pour l'orienteur rowId 6
curl "https://n8n.incubateur.dnum.din.developpement-durable.gouv.fr/webhook/occ-generate?rowId=6"
```

Réponse :

```
{
  "rowId": 6,
  "token": "6.d4f8a2...",
  "url": "https://stiiig.github.io/grist-widgets/widgets/emile/validation-compte?token=6.d4f8a2..."
}
```

Envoyer le champ `url` par email à l’orienteur. Quand il clique, la page `validation-compte` appelle OCC-VALIDATE qui passe `Compte_valide → "Oui"` .

Flux complet OCC

```
creation-compte-orienteur (frontend)
    │
    │  AddRecord ACCOMPAGNANTS → Compte_valide = "En attente"
    ▼
Proxy n8n POST → Grist (nouveau rowId)
```

```
    │
    │  GET /webhook/occ-generate?rowId=X
    ▼
Workflow OCC-GENERATE
    │  Génère token = rowId.HMAC
    │  Respond { rowId, token, url }  ← frontend affiche le lien
    │  PATCH ACCOMPAGNANTS.Lien_validation = url  (background)
    ▼
Email envoyé à l'orienteur avec le lien validation-compte?token=X.HMAC
    │
    │  (orienteur clique sur le lien)
    ▼
Page validation-compte (frontend)
    │
    │  GET /webhook/occ-validate?token=X.HMAC
    ▼
Workflow OCC-VALIDATE
    │  Vérifie HMAC
    │  GET ACCOMPAGNANTS record (vérifie Compte_valide)
    │  PATCH Compte_valide = "Oui"
    │  Respond { status: "ok", nom: "..." }
    ▼
Page affiche "Compte activé !" + bouton "Inscrire un·e candidat·e"
```

Table IDs Grist

> ⚠️ Le **Table ID** Grist (utilisé dans l'API et dans le code) est différent du **nom d'affichage** visible dans l'onglet. En cas de renommage d'une table, le Table ID ne change pas automatiquement.

Tables utilisées par les widgets EMILE :

Table ID (API)	Champs clés	Utilisée par
`CANDIDATS`	`Reference` (identifiant dossier), `Responsable_candidat` (Ref → ACCOMPAGNANTS), `Date_de_naissance` (source âge — ne pas utiliser `$Age`)	`fiche-candidat`, `inscription-candidat`, `liste-candidats`
`ETABLISSEMENTS`	—	`ajout-etablissement`, `creation-compte-orienteur`
`ACCOMPAGNANTS`	`Compte_valide` (`"En attente"` / `"Oui"`), `Lien_validation` (texte — magic link OCC)	`creation-compte-orienteur`, `inscription-candidat`
`DPTS_REGIONS`	`Nom_departement`, `Numero`, `Nom_region`, `Territoire_depart`	`ajout-etablissement`, `fiche-candidat`
`_grist_Tables`	—	`loadColumnsMetaFor` (méta interne)

Table ID (API)	Champs clés	Utilisée par
`_grist_Tables_column`	—	`loadColumnsMetaFor` (méta interne)

Mettre à jour la clé API Grist

1. Aller sur Grist → Profile Settings → API Key → copier la clé
2. Dans n8n : **Credentials** → **Bearer Auth Grist** → coller la nouvelle clé
3. Aucun redéploiement nécessaire

Limitations connues

~~AddRecord / UpdateRecord — preflight CORS bloquant~~ — ✅ Résolu

Les actions `AddRecord` et `UpdateRecord` sont opérationnelles en mode REST. La technique : `rest.ts` envoie toutes les écritures avec `Content-Type: text/plain;charset=UTF-8` (simple CORS request, pas de preflight) et un champ `_action` dans le body JSON pour que n8n distingue les deux opérations. Le workflow POST a été mis à jour avec un nœud IF pour router selon le Content-Type.

Pièces jointes (AttachmentField)

Fonctionnalité	Disponible	Condition
Affichage des noms	✅	Automatique (`_grist_Attachments` proxifié via workflow GET)
Téléchargement	✅	Branche `?attachId=X` dans le workflow GET configurée
Upload	✅	Workflow POST configuré
Suppression	✅	Upload opérationnel (ré-enregistre la liste sans l'ID supprimé)

Fiche candidat — onglets non mappés

Seul l'onglet "Administratif" de `fiche-candidat` est mappé sur des colonnes Grist. Les autres onglets affichent "non mappé" hors iframe.

Expiration des magic links

Les tokens sont permanents (pas d'expiration). Pour révoquer un lien, il faudrait changer le secret HMAC (invalide tous les tokens existants). Une expiration par date pourrait être ajoutée ultérieurement.